

---

# When Waiting Is Not a Retry: A Bounded Study of Observation Accounting and Dispatch Selection

---

## Abstract

A scheduler can observe unfinished work without encountering an execution failure. Charging every such observation to a retry budget can terminate a task before it returns a result; repeatedly selecting that task can also exclude independent ready work. We compare two frozen SQLite-backed scheduler implementations under seven matched, controlled-clock scenarios and an unchanged twelve-method contract suite. An opt-in candidate refunds the provisional attempt charge only for an accepted pending return, limits waiting through separate poll and elapsed-time budgets, and rotates selection through a persisted cursor. With one allowed attempt, the baseline fails on the first pending return, whereas the candidate accepts four pending returns and completes on dispatch five. In a separate six-turn trace, the baseline dispatches only the pending task; the candidate completes an independent task on turn two. Wait-budget exhaustion blocks the candidate without reporting success, and explicit retries still exhaust its attempt budget. These results characterize local transition behavior on deterministic fixtures, not production latency or general starvation freedom.

## 1 Introduction

A bounded task scheduler must distinguish failure from lack of completion. Consider an adapter that observes an already owned job and reports that its result is not yet available. If a task has one allowed attempt, treating this observation as a spent attempt can make the task fail immediately, even though the adapter has reported no execution failure. A different problem arises when the task remains ready for observation: a scheduler that scans from the beginning of a fixed task list can repeatedly choose it before an independent task. Increasing the attempt limit alone does not address either semantic distinction.

This paper asks two concrete questions: when should a pending observation consume an attempt, and can repeated pending work exclude an independent ready task? We study a local task-DAG runtime with durable SQLite state, trusted handlers, and at most one dispatch per scheduler tick. The comparison is between a frozen baseline and a candidate snapshot that introduces explicit waiting bounds and rotating task selection [3]. The task plans and handler responses are controlled so that differences in transitions can be inspected directly. The experiment concerns these implementations under a specified observation contract; it does not infer whether any real external job is healthy merely from a pending label.

The candidate separates two decisions. Observation accounting determines whether a handler return spends a bounded execution attempt. Selection determines which eligible task receives the next dispatch. Under an explicit wait policy, a successfully returned and accepted pending result increments a poll counter and refunds the provisional attempt charge. The elapsed waiting interval and number of observations have independent finite limits. Selection uses a persisted cursor, while dependencies,

deadlines, authorization, and terminal states still constrain eligibility. A wait limit therefore does not become permission to declare completion, and rotation does not make a blocked task ready.

Our contribution is a bounded empirical characterization of this separation. First, a matched completion trace shows premature exhaustion under legacy accounting and completion after four pending observations under the candidate. Second, a competing-ready-task trace exposes finite exclusion under fixed-order selection and second-turn service under rotation. Third, the remaining scenarios and the contract suite establish the tested boundaries: wait exhaustion blocks, explicit retry outcomes still consume attempts, cancellation stops dispatch, and plans without a wait policy retain legacy accounting. The study offers a small reproducible regression target and an operational interpretation of its counters. It does not introduce a new scheduling theory or claim a broad reliability improvement.

## 2 Observation and dispatch semantics

### 2.1 Study object and terminology

A plan supplies an ordered list of tasks and their dependencies. Each task has a finite maximum attempt count  $A$ , an estimated duration, a risk label, and completion conditions. A trusted host supplies the handler and authorization decision. The handler returns one of `complete`, `pending`, `retry`, `blocked`, or `failed`. A *pending observation* is a normally returned report that work remains unfinished; a *retry* requests another execution opportunity after the current attempt did not complete. Neither response establishes successful completion. A `complete` return is accepted only when it supplies evidence that the handler’s verifier accepts.

The persisted task status is distinct from the handler outcome. Pending and retry normally map back to `todo`; verified completion maps to `done`. A task is eligible for dispatch when it is `todo`, all dependencies are `done`, its next-dispatch deadline has arrived, a handler is available, authorization permits execution, and applicable wait limits have not been exhausted. One tick can claim at most one task. Duplicate event identifiers are rejected, and active leases constrain a claim’s validity.

We use *attempt counter* for the implementation field `attempts`. It is not exactly a count of failures: the runtime increments it provisionally on every claim, and a successful completion also retains that increment. Thus the completed candidate task in our first scenario has `attempts=1`. The relevant distinction is whether pending observations spend this limited counter, not whether every execution leaves it at zero. We use *dispatch count* for actual handler invocations and *poll count* for accepted opted-in pending returns. These quantities must not be interchanged.

### 2.2 Separate bounds for attempts and waiting

An explicit wait policy contains a maximum pending count  $P$ , an elapsed limit  $T$ , and a diagnosis threshold  $D$ . The candidate requires integer  $1 \leq P \leq 100000$ , finite  $1 \leq T \leq 604800$  seconds, and finite  $1 \leq D \leq T$  seconds; booleans are not accepted as numeric bounds. The implementation also requires  $1 \leq A \leq 20$ . These are source-defined guard ranges, not experimentally optimized parameters.

Let  $a$  and  $p$  be the task’s attempt and pending counters before a claim. When the current claim remains valid and its result is accepted, accounting on the tested paths is

$$(a', p') = \begin{cases} (a, p + 1), & \text{pending with an explicit wait policy,} \\ (a + 1, p), & \text{retry or verified completion.} \end{cases} \quad (1)$$

The first branch is implemented by incrementing  $a$  at claim time and decrementing it after a pending return. It is not a blanket exemption for attempts that crash, lose their lease, or have unknown side effects. Without an explicit wait policy, pending also retains the increment, and a return to `todo` with  $a' \geq A$  becomes `failed`.

The first accepted pending return sets  $t_0$ , using the tick’s claim-time clock value. After pending returns, and before subsequent claims, the candidate tests

$$p \geq P \quad \text{or} \quad t - t_0 \geq T. \quad (2)$$

If either condition holds, the task becomes `blocked` with reason `kind wait_budget` and requests diagnosis. The pre-claim check prevents another poll once elapsed time is exhausted; the post-return

check catches the observation that reaches  $P$ . In this snapshot, the counters and waiting origin persist across later retries: they are cumulative for the task, rather than automatically restarted as a new wait episode.

The diagnosis threshold is weaker than a termination limit. Once  $t - t_0 \geq D$ , a tick sets `diagnosis_required` even if the next observation is not yet due. It does not assert that the task has failed or that a replacement has succeeded. This distinguishes three facts that a single counter would conflate: an execution attempt was spent, an observation budget was exhausted, or host diagnosis became due.

### 2.3 Rotating selection with durable eligibility

The baseline scans task order from its first element on every tick. If the first task returns pending and its deadline is due again at the next tick, it can be selected again while a later independent task remains ready. The candidate stores a cursor  $c$  in the same durable state. For a list of  $n$  tasks it scans indices  $(c + j) \bmod n$  for  $j = 0, \dots, n - 1$ , claims the first eligible task at index  $i$ , and persists  $c \leftarrow (i + 1) \bmod n$  with the claim. No eligible task means no dispatch. Selection can still require an  $O(n)$  scan; our measurements do not quantify its execution cost.

The cursor advances on a claim, not on verified progress. Consequently, a pending return does not force the next scan to restart at the same task. Persisting the cursor matters because recreating an engine object should not erase the next starting position. The runtime uses SQLite transactions for claims and state updates, then executes the handler outside that transaction. A result is committed only while the token, task status, lease, and cancellation state still permit it. A stable key derived from the immutable plan and task identity accompanies dispatches. These mechanisms explain the tested restart behavior; they do not by themselves establish exactly-once external effects or correctness under every concurrent interleaving.

## 3 Experimental design

**Frozen implementations.** The baseline corresponds to commit `e8ee3dca5dc8cd076fc031dd4510dabde214e046`. The candidate is an uncommitted working-tree snapshot identified by file hash, not by a release or the baseline commit. Both actual core implementations are included in the study artifact [3]; we do not replace them with a model of the transition rules. The baseline accepts the extra `wait_policy` plan key but continues its legacy accounting. Feeding the same plan to both versions therefore compares actual behavior without assuming that the baseline implements the new contract.

**Controlled execution.** The supplied prospective protocol and reproducer [1] run seven scenarios against each variant, producing fourteen SQLite databases and six tick states per scenario:  $7 \times 2 \times 6 = 84$  recorded states. Each state includes cumulative dispatch records. A controlled clock starts at 1000 seconds and normally advances by one second per tick; the elapsed-limit scenario advances by 100 seconds on its second tick. The plan’s minimum and maximum supervision intervals are both one second. These values are inputs to transition tests, not measurements of elapsed host time. A fresh engine object is constructed for each tick over the persisted store.

The handler is idempotent, requires no external capability, and is always authorized by the fixture. It normally returns pending. In the completion scenario it returns verified completion after four pending invocations; the independent quick task completes whenever dispatched. In the retry scenario it returns pending once and retry thereafter. Verification accepts the fixture evidence token checked. Thus completion evidence exercises the local verifier branch but does not certify an external artifact. Cancellation is applied immediately before the second tick in its scenario.

The default observation limits are  $P = 10$ ,  $T = 100$  seconds, and  $D = 20$  seconds. The poll-limit scenario changes  $P$  to two. Attempts are limited to two except in the completion and legacy scenarios ( $A = 1$ ) and the competing-task scenario ( $A = 20$ ). The latter keeps both versions within their attempt budget across the six observed turns, so premature attempt exhaustion does not explain the absence of service to the quick task. Both tasks have no dependencies and the slow task appears first.

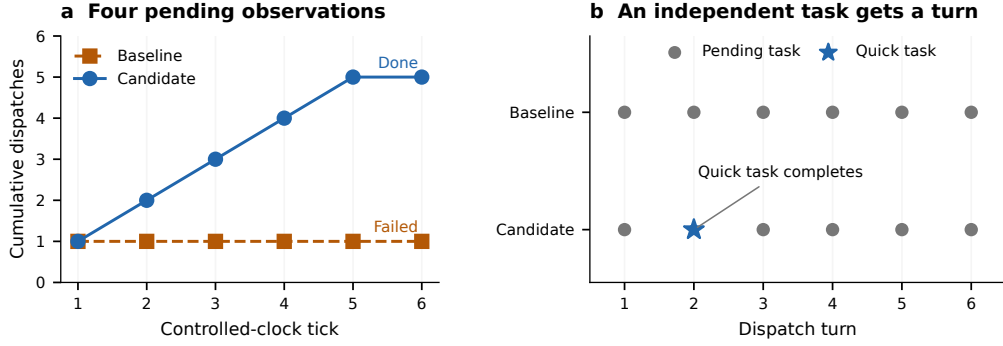


Figure 1: Two distinct controlled-clock scenarios from the frozen implementations. (a) With  $A = 1$ , the baseline fails after its first pending return; the candidate accepts four pending returns and completes at dispatch five. Lines connect discrete recorded tick states and do not interpolate execution times. (b) With  $A = 20$ , the baseline dispatches the slow pending task on all six turns; the candidate completes the independent quick task on turn two. All seven scenarios are retained in Table 1. The figure shows transition and service counts, not latency or a starvation-freedom proof.

**Measurements and contract checks.** We report statuses, counters, dispatch identities, and reasons. The unit of observation is a deterministic scenario execution, with its six tick states representing a trajectory, not six independent samples. We do not estimate sampling uncertainty or attach confidence intervals to these counts. The supplied run used Python 3.12.14 on x86-64 Linux, with local SQLite and no external job runner. An unchanged twelve-method `unittest` suite additionally checks validation, repeated/early events, restart, stable keys, diagnosis, cancellation, and budgets. This is a candidate-contract suite: baseline failures against newly introduced fields must be distinguished from behavioral comparisons.

## 4 Results

### 4.1 Pending need not spend a failure opportunity

With one allowed attempt, the baseline fails at the first pending return and never invokes the handler again in the six-tick window (Figure 1a). The candidate’s first four pending returns leave  $a = 0$  while increasing  $p$  from one to four. Its fifth dispatch returns completion; evidence verification succeeds, the task and run become done, and  $a = 1$ . The sixth tick makes no additional dispatch. This contrast directly answers the accounting question for an adapter that successfully reports unfinished work under the explicit policy.

Keeping observations out of  $a$  does not remove the failure bound. In the pending-then-retry scenario, the baseline spends its first attempt on pending and fails at its second dispatch after one retry. The candidate spends no attempt on the initial pending return, then spends one attempt on each of two retries and fails on dispatch three. The final attempt counts are both two, but their meanings differ: the baseline has charged one observation and one retry, while the candidate has charged two retries. No claim of eventual completion follows from the refund rule.

### 4.2 Repeated pending work can exclude a ready task

The competing-task scenario contains a permanently pending slow task followed by an independent quick task. Across six ticks, the baseline makes six slow-task dispatches, leaving the quick task todo. Its attempt count reaches six, well below  $A = 20$ ; this finite exclusion therefore occurs while both tasks could still be serviced. The candidate dispatch sequence is slow, quick, slow, slow, slow, slow (Figure 1b). The quick task completes on turn two, while the slow task remains todo with five pending observations and zero spent attempts after turn six. The candidate run remains active because the slow task is unfinished.

| Scenario                    | Baseline outcome                   | Candidate outcome                       |
|-----------------------------|------------------------------------|-----------------------------------------|
| Four pending, then complete | Failed; 1 dispatch, $a = 1$        | Done; 5 dispatches, $a = 1, p = 4$      |
| Independent quick task      | Quick todo; 6 slow dispatches      | Quick done on turn 2; 5 slow dispatches |
| Two-poll wait limit         | Failed; 2 dispatches, $a = 2$      | Blocked; 2 dispatches, $a = 0, p = 2$   |
| 100-second wait limit       | Failed; 2 dispatches, $a = 2$      | Blocked before poll 2; $a = 0, p = 1$   |
| Legacy, no wait policy      | Failed; 1 dispatch, $a = 1$        | Failed; 1 dispatch, $a = 1$             |
| Pending, then retries       | Failed after 1 retry; 2 dispatches | Failed after 2 retries; 3 dispatches    |
| Cancel after observation    | Cancelled; 1 dispatch              | Cancelled; 1 dispatch                   |

Table 1: All seven matched scenarios, observed over six ticks each. Dispatch totals and  $a, p$  refer to the slow task except where explicitly described otherwise; the competing-task row reports the quick task’s status. Candidate blocked outcomes have reason kind `wait_budget` and request diagnosis. A blocked task is neither failed nor successfully completed. Missing baseline poll counters are not interpreted as zeros.

This is evidence about service order, not a throughput comparison. Both versions make six dispatches in this scenario, but the identity and usefulness of one dispatch differ. The code’s cursor rule explains the observed second turn: after selecting index zero, selection begins at index one. The experiment neither establishes an infinite starvation trace for the baseline nor proves starvation freedom for arbitrary candidate workloads. The baseline’s finite attempt limit may eventually stop the slow task; the observed six-turn exclusion remains a distinct defect relative to the tested second-turn contract.

### 4.3 Bounds, compatibility, and negative outcomes

The poll-limit scenario shows why an observation exemption must have a separate finite bound. On its second pending return, the candidate reaches  $p = 2$  and becomes `blocked`, retaining  $a = 0$  and requesting diagnosis. Subsequent ticks do not dispatch it. The baseline instead reaches its two-attempt limit and fails. In the elapsed-limit scenario, the candidate blocks at controlled time 1101, exactly 100 seconds after the first pending claim at 1001. It makes only one dispatch because the limit is checked before another poll. The baseline makes a second dispatch and fails through attempt exhaustion. Neither candidate outcome asserts that the external work completed or that a timeout proves an execution failure.

Two unchanged outcomes constrain compatibility claims (Table 1). Without a wait policy, both implementations fail after the first pending return when  $A = 1$ . Explicit cancellation after the first observation prevents a second dispatch in both. These results support compatibility for the tested accounting and cancellation paths; they do not show identical scheduling for all legacy multi-task plans, because the candidate’s cursor rotation is global rather than restricted to tasks with wait policies.

The candidate passes all twelve methods in the unchanged contract suite. The baseline passes the cancellation and legacy methods and reports 15 failure records plus four errors across the suite’s twelve methods. The 15 failures include parameter subtests; these are not 15 independent experiments, and the combined total is not 19 tests. The four errors concern missing candidate fields such as `pending_polls` and `diagnosis_required`. Behavioral conclusions therefore rest on the matched trajectories as well as the source, rather than treating a missing field as direct evidence of a scheduling failure. The candidate’s restart test preserves the slow–quick–slow order and the slow task’s stable key across reopening the store. Diagnosis tests show a flag can be raised before another poll is due, while early and duplicate wakes do not add an observation.

## 5 Relation to prior work and scope

The closest comparison in this study is the frozen predecessor of the same task runtime [3]. It already supplies durable state, bounded attempts, leases, authorization, and completion verification. The candidate changes the interpretation of pending returns, adds finite observation bounds and diagnosis state, and rotates the scan origin. These are the specific deltas examined here; the experiment does not attribute every runtime property to the candidate.

Scheduling in other systems can address a different resource and correctness problem. GPipe [2], for example, partitions a neural network across accelerators and pipelines micro-batches; its Section 2.2 and Figure 2 describe the execution schedule, and Section 3 evaluates performance. That work concerns coordinated model computation. Our ready-task trace instead tests whether a local observer occupies consecutive dispatch opportunities. We borrow neither its performance results nor its semantics as evidence for retry handling. The supplied literature supports this distinction but is not a comprehensive survey of workflow schedulers, fair queues, or retry systems; novelty relative to those broader literatures remains unestablished.

The study’s small scale is useful for exposing a counterexample to the assumption that every dispatch should consume the same retry budget. It is insufficient for deployment claims. Handlers return immediately under deterministic control, so there is no measured production latency, wall-clock speedup, external adapter failure distribution, or concurrency stress. The verifier accepts a fixed token, and no model, server job, or real waiting process runs. The evidence therefore isolates scheduler transitions rather than autonomous work quality or end-to-end reliability.

The intervention also changes accounting and selection together. The completion trace contains only one task, and the six-turn competing-task trace keeps both variants below their attempt limit, making the two intended effects interpretable. Nevertheless, no factorial ablation toggles each change independently, and the study does not estimate their separate latency contributions. More elaborate workloads could couple eligibility, delay, failure, and observation behavior. A general liveness argument would need explicit assumptions about recurring wakeups, task-list stability, continuing eligibility, handler termination, and concurrent claims; the finite trace proves none of these assumptions.

Finally, a pending result is only as trustworthy as the adapter that returns it. The candidate refunds an accepted normal return, not an unobserved crash or an ambiguous effect. Repeated incorrect pending labels can still delay error recognition until a wait bound is reached. Choosing useful  $P$ ,  $T$ , and  $D$  values requires information about actual work durations and observation costs that this experiment does not provide. A deployment evaluation would need realistic handlers, contention and restart faults, matched legacy workloads, and an explicit fairness specification before asserting reliability or performance improvements.

## 6 Conclusion

For the tested contract, a pending observation should not spend an execution attempt when a trusted handler successfully reports unfinished work under explicit, finite waiting limits. It should spend a separate observation allowance; retry returns should continue to spend attempts, and wait exhaustion should block for diagnosis rather than imply completion. The supplied implementations demonstrate why the distinction matters: four pending observations can precede completion without exhausting a one-attempt task, while the legacy behavior fails immediately. They also show that accounting alone is not the service-order question. In the observed competing-task trace, fixed-order selection excludes a ready task for six turns, whereas a persisted rotating cursor gives it turn two. These are concrete local outcomes and regression targets, with general fairness and production behavior left open.

## References

- [1] Fixture protocol. Bounded observation and dispatch semantics: Prospective protocol and controlled-clock fixtures, 2026. Study artifact, `study/protocol.md` (section S), `study/reproduce.py`, and `study/runs/run01/systems`. Recorded 6 October 2026.
- [2] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL [https://proceedings.neurips.cc/paper\\_files/paper/2019/file/093f65e080a295f8076b1c5722a46aa2-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2019/file/093f65e080a295f8076b1c5722a46aa2-Paper.pdf).
- [3] Runtime snapshot. Frozen local task runtime: Baseline and candidate core snapshots, 2026. Study artifact, `study/systems`. Baseline commit `e8ee3dca5dc8cd076fc031dd4510dabde214e046`; candidate working-tree bytes identified by SHA256 in `source_versions.json`.

## A Reproduction details

The artifact retains the two core snapshots, the unchanged waiting suite, the prospective protocol, the supplied run's full traces and fourteen databases, and the runner. Core SHA256 digests are given in the accompanying source manifest; their prefixes are b5047fd679b1aac2 (baseline) and 47dd90f76e42679e (candidate). The systems runner imports only these frozen cores and creates a fresh run directory. From `work/study`, the command is:

```
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 \  
python reproduce.py systems --run-id systems-replay-01
```

A new run identifier is required on each execution. Random claim tokens and temporary paths mean that replay validation should compare defined semantic fields rather than require byte-identical SQLite files. The figure's editable plotting source and its trace inputs are included with the paper; the paper README gives its separate rendering and document-build commands. The recorded host CPU count is nine, but the study is not a parallel CPU benchmark. No training data, participant data, or learned model measurements are involved.